

Staffing Intelligence Platform

A locally-hosted, open-source system for placement intelligence, workforce risk, and account analytics

15 Features delivered across 5 modules	~9k Codebase lines of Python	0 External services fully self-hosted	9 Build time fortnightly sprints
---	---	--	---

This document describes the design, implementation, and operational characteristics of an in-house staffing intelligence platform. It is written for engineers and data scientists who will extend or operate the system, and for technical stakeholders evaluating its architecture. It covers what was built, why each design decision was taken, how the models behave, and — importantly — where the current implementation falls short of production readiness.

Contents

1 Executive summary

2 Problem definition

- 2.1 Operational context
- 2.2 Failure modes in current practice
- 2.3 Scope and non-goals

3 System architecture

- 3.1 Layer model
- 3.2 The dependency chokepoint
- 3.3 Technology selection and trade-offs

4 Data architecture

- 4.1 Relational schema
- 4.2 Storage tier rationale
- 4.3 The polymorphic prediction store

5 The matching engine

- 5.1 Composite scoring model
- 5.2 Sub-scorer definitions
- 5.3 Weight calibration
- 5.4 Skill gap severity

6 Predictive models

- 6.1 Submission success
- 6.2 Contractor attrition
- 6.3 Client churn
- 6.4 Timesheet anomaly detection
- 6.5 Revenue forecasting
- 6.6 Rate optimisation

7 NLP and language model layer

- 7.1 Resume parsing pipeline
- 7.2 Embedding strategy
- 7.3 Retrieval-augmented assistant

8 Engineering practices

- 8.1 Observability
- 8.2 Performance instrumentation
- 8.3 Test strategy
- 8.4 Scheduled work

9 Deployment and operations

- 9.1 Runtime topology
- 9.2 Resource requirements
- 9.3 Failure modes encountered

10 Evaluation and limitations

- 10.1 What the metrics do and do not mean
- 10.2 Known gaps
- 10.3 Fairness and compliance considerations

11 Roadmap

A Appendix — data dictionary

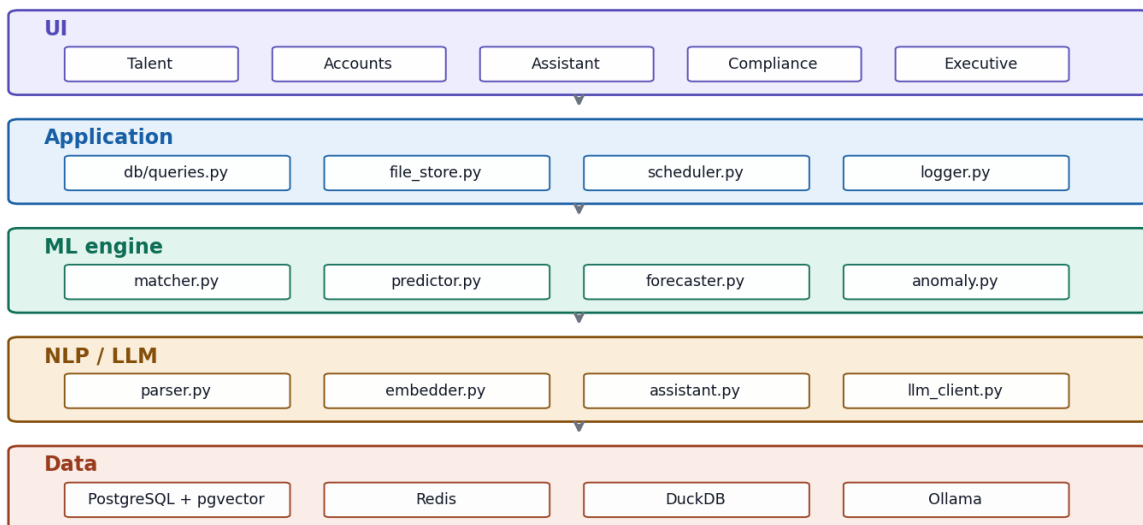
1 Executive summary

Staffing firms operate on thin margins and short decision windows. A recruiter deciding which three candidates to submit against a requisition is making a judgement that determines both placement probability and account profitability, usually in under an hour, usually from a résumé pile that no one has time to read properly. The same firm simultaneously carries workforce risk it cannot see: contractors drifting toward resignation, accounts quietly sliding below margin, visa clocks running down.

This platform addresses those problems as a single system rather than a collection of reports. Fifteen features span five functional modules — talent intelligence, account analytics, recruiter productivity, compliance, and executive reporting — over a shared data model. Every component runs on the operator's own hardware. There are no external API calls, no per-seat licensing, and no candidate data leaving the building.

What was built

The system decomposes into six layers, each depending only on the layer beneath it. A composite matching engine scores candidate–requisition fit across six weighted dimensions, combining dense-vector semantic similarity with hard eligibility constraints. Four supervised models predict submission outcome, contractor attrition, account churn, and timesheet irregularity. A résumé parser converts unstructured PDF and DOCX files into structured candidate records. A retrieval-augmented assistant answers natural-language questions over the live database using a locally-hosted language model.



Everything below the UI is framework-agnostic — swapping Streamlit touches none of it

Figure 1 — Layer model. Each layer depends only on the one below. The UI holds no business logic, which is what makes it replaceable without touching the rest of the system.

What this document is honest about

The four supervised models currently ship trained on synthetically generated feature distributions. Their cross-validated AUC figures — reported in section 6 and ranging from 0.91 to 0.96 — measure how faithfully each model recovered the rules of the data generator that produced its training set. They are **not** estimates of real-world predictive performance, and quoting them as such would be misleading. Section 10 sets out precisely what would need to happen before these numbers carry operational meaning.

Reading this document

Sections 3 through 7 describe design and implementation. Section 10 is the critical evaluation and should be read before any decision to rely on model output operationally. Readers looking only for setup instructions should use the repository README rather than this paper.

2 Problem definition

2.1 Operational context

A contract staffing desk runs a repeating cycle. A client issues a requisition. Recruiters search the internal candidate pool and external boards, shortlist a handful, and submit them. The client interviews, possibly extends an offer, and the placed contractor begins billing. Revenue is the spread between the bill rate charged to the client and the pay rate owed to the contractor, accrued hourly for the assignment's duration.

Three properties of this cycle shape the system design. First, speed dominates: on a competitive requisition the first credible submission frequently wins, so a matching process that takes minutes rather than hours has direct commercial value. Second, the economics are margin-sensitive: a placement at 12% margin consumes the same recruiter effort as one at 28%, so mispricing is expensive in a way that is easy to miss in aggregate reporting. Third, a placed contractor is a continuing liability as well as an asset — an unexpected resignation triggers an emergency backfill under worse conditions than the original search.

2.2 Failure modes in current practice

Failure mode	Mechanism	Consequence
Keyword-only matching	Applicant tracking systems match literal strings. A candidate listing "ReactJS" does not surface for a "React" requirement.	Qualified candidates never reach the shortlist
Undifferentiated submissions	No signal separates a strong submission from a weak one before the client responds.	Recruiter effort spread evenly across unequal prospects
Invisible attrition	Warning signals — communication gaps, overtime spikes, declining feedback — are recorded in separate systems and never correlated.	Backfills begin only after resignation
Reactive account management	Declining requisition volume is noticed at quarterly review.	Intervention comes after the client has already disengaged
Margin drift	Rates are negotiated per placement without reference to a market baseline.	Systematic underpricing that compounds across an account
Manual compliance tracking	Visa and I-9 expiry dates live in spreadsheets.	Deadline discovered late; contractor pulled off billing

2.3 Scope and non-goals

The platform is scoped as an **internal decision-support tool for a single firm**. That framing drove several architectural decisions and excluded others.

In scope	Explicitly out of scope
Single-tenant deployment on operator hardware	Multi-tenant SaaS with row-level isolation
Decision support — ranked lists, risk scores, flags	Automated decisions taken without human review
Read-mostly analytics over an existing data model	Replacing the applicant tracking system of record
Locally-hosted language model inference	Third-party model APIs of any kind
Dataset scale in the thousands	Horizontal scale-out; distributed query execution

A deliberate constraint

No candidate data, résumé text, or client information is transmitted to any external service. This rules out hosted embedding APIs and commercial language models, and it directly determined the choice of sentence-transformers and Ollama over higher-performing hosted alternatives. The performance cost is real and is quantified in section 7.

3 System architecture

3.1 Layer model

The system is organised into six layers with strictly downward dependencies. No layer imports from a layer above it, and no layer skips a level to reach two below. This is enforced by convention rather than tooling, but it is verifiable by inspecting imports.

Layer	Responsibility	Key modules
Presentation	Rendering, user input, session state. Contains no business logic.	app.py, pages/*.py, app/theme.py
Application	Data access, file handling, scheduling, logging. The only layer that opens database sessions.	db/queries.py, data/file_store.py, data/scheduler.py
ML engine	Scoring, prediction, forecasting, anomaly detection.	ml/matcher.py, ml/predictor.py, ml/forecaster.py, ml/anomaly.py
NLP / LLM	Document parsing, embedding generation, retrieval, text generation.	ml/parser.py, ml/embedder.py, ml/assistant.py, ml/llm_client.py
Persistence	Relational storage, vector index, cache, analytical engine.	PostgreSQL + pgvector, Redis, DuckDB
Infrastructure	Container orchestration, migrations, model inference runtime.	docker-compose.yml, Alembic, Ollama

3.2 The dependency chokepoint

The single most consequential design decision was routing all database access through one module, `db/queries.py`. No presentation-layer file constructs a query, opens a session, or imports an ORM model. Every page calls a named function that returns a pandas DataFrame.

```
# A page never does this:
session = get_session()
rows = session.execute(select(Candidate).where(...)).scalars().all()

# It does this:
from db.queries import get_candidates
candidates = get_candidates(active_only=True)
```

This costs a little expressiveness — a page needing an unusual filter must add a function rather than write an ad-hoc query — and it returned that cost several times over:

- **Caching was added without touching any page.** Redis memoisation was introduced late in development. Because every page already called through the same functions, the change was localised.
- **The presentation layer is genuinely replaceable.** Substituting Streamlit for a different framework requires reimplementing roughly 1,400 lines of view code and nothing else. The 85% of the codebase below the UI is unaware of which framework is rendering it.
- **Schema changes have one blast radius.** When the attrition signal columns were added, exactly one function needed updating.
- **Query performance is observable in one place.** The logging decorator wraps query functions, so every slow query surfaces without page-level instrumentation.

The cost of getting this wrong

During development an attrition page failed because `get_candidates()` returned a fixed column list that omitted four signal fields. The fix was a single-line change in one file. Had each page built its own query, the same defect would have been reproduced independently across every page reading candidate data.

3.3 Technology selection and trade-offs

Selection was constrained by three requirements: fully open-source, operable on a single machine, and maintainable by a Python developer without frontend specialisation. The table records what was chosen, what was rejected, and what the choice costs.

Component	Selected	Rejected alternative	Trade-off accepted
UI framework	Streamlit	React + FastAPI	Less layout control and a full-page rerender model, in exchange for roughly 4× faster delivery and no JavaScript
Relational store	PostgreSQL 16	SQLite	Requires a running service, but provides array columns, JSONB, and the pgvector extension
Vector index	pgvector	Qdrant / FAISS	Lower ceiling at very large scale; removes an entire service and keeps vectors transactionally consistent with their rows
Embeddings	all-MiniLM-L6-v2	OpenAI text-embedding-3	Measurably weaker retrieval, but satisfies the no-external-service constraint and runs on CPU
Tabular models	XGBoost / LightGBM	Neural networks	None material. Gradient boosting outperforms deep models on tabular data at this sample size
Analytics	DuckDB	ClickHouse	Single-node only; eliminates a service and reads Postgres directly with no ETL
LLM runtime	Ollama	vLLM / hosted API	Lower throughput; trivial single-machine operation and no data egress

On choosing pgvector over a dedicated vector database

A separate vector store is the conventional choice and would scale further. It was rejected because it introduces a consistency problem: two systems must be kept in agreement about which candidates exist. With pgvector the embedding is a column on the row it describes, so a deleted candidate cannot leave an orphaned vector behind. At this data scale the performance difference is immaterial; the correctness difference is not.

4 Data architecture

4.1 Relational schema

Nine tables model the domain. The schema is deliberately normalised — denormalisation for read performance was considered and rejected, since DuckDB handles the analytical aggregations that would have motivated it.

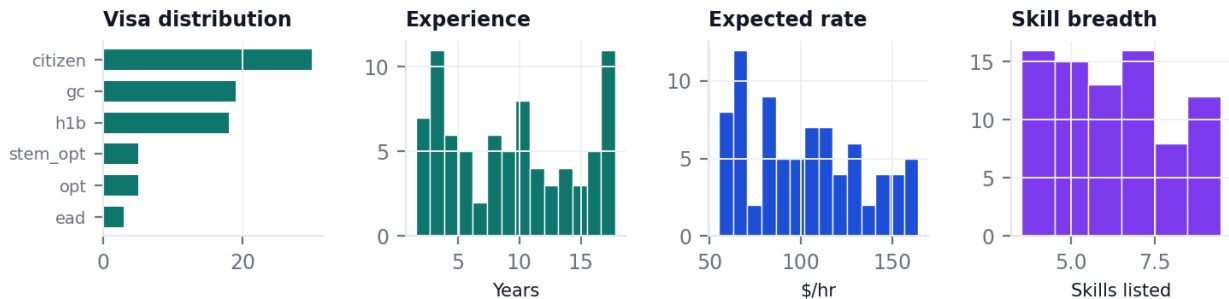


Figure 2 — Distribution characteristics of the seeded reference dataset (80 candidates). Visa mix, experience, rate expectation, and skill breadth were generated to approximate a realistic contract IT desk rather than uniform random values, so that model behaviour during development resembled production conditions.

Two tables carry disproportionate structural weight. The `placements` table is the central join — it connects candidate, requisition, recruiter, and client, and every funnel metric, conversion rate, and margin calculation derives from it. The `candidates` table is the second hub, feeding timesheets, payroll, embeddings, and three of the four predictive models.

4.2 Storage tier rationale

Four storage technologies are in use. Each earns its place by doing something the others do poorly.

Tier	Holds	Durability	Why not Postgres
PostgreSQL + pgvector	All business entities; 384-dimensional embeddings on candidates and jobs	Permanent	—
Redis	Ranked match results, assistant conversation state, scheduler timestamps	Ephemeral, TTL 3 min – 2 days	Writing transient cache entries to a durable store creates vacuum pressure and offers no benefit
DuckDB	Funnel aggregates, recruiter KPIs, margin analysis, bench cost	Rebuilt nightly	Analytical scans over placement history are columnar workloads; DuckDB reads Postgres directly, so no ETL is required
Local filesystem	Résumé documents, trained model artefacts, RAG index, logs	Permanent, regenerable	Storing binary documents in a relational database complicates backup and offers nothing here

Résumé files illustrate the division: the document itself sits on disk, the extracted plain text sits in a `TEXT` column, the derived embedding sits in a `vector(384)` column, and the relational row stores only the path. Each representation lives where it is cheapest to query.

4.3 The polymorphic prediction store

Model outputs are written to a single table using an entity-type discriminator rather than one table per model:

```

predictions
id          integer  PK
entity_type varchar  'candidate' | 'client' | 'placement'
entity_id   integer  FK by convention, not constraint
model_name  varchar  'submission_success' | 'attrition_risk' | ...
score       float   model output
features    jsonb    inputs used, for explainability

```

```
created_at    timestamp for drift analysis
```

The trade-off is explicit. Referential integrity is lost — the database cannot enforce that `entity_id` refers to a real row, because the target table varies by `entity_type`. In exchange, adding a fifth or sixth model requires no migration, and drift analysis across all models is a single query rather than a union across per-model tables. Given that models are expected to be added regularly and referential integrity on an audit log is a weak requirement, the exchange favours flexibility.

Storing the feature vector alongside the score is what makes the record useful beyond audit. When a prediction is later found to have been wrong, the inputs that produced it are recoverable without reconstructing the state of the database at prediction time.

5 The matching engine

5.1 Composite scoring model

Candidate–requisition fit is scored as a weighted linear combination of six independently computed sub-scores, each normalised to the unit interval:

$$\text{match}(c, j) = 100 \times \text{SUM}[w_i * s_i(c, j)], \quad i = 1..6$$

subject to $\text{SUM}[w_i] = 1$ and $0 \leq s_i \leq 1$

A linear model was chosen over a learned ranker for one reason: at deployment there was no labelled outcome data. A learning-to-rank model needs historical submissions with known results, and the firm's existing records did not include the negative examples — candidates considered and rejected — that such a model requires. A transparent weighted sum can be reasoned about, defended to a recruiter who disagrees with a ranking, and tuned by hand. Section 11 sets out the migration path to a learned ranker once the platform has accumulated its own outcome labels.

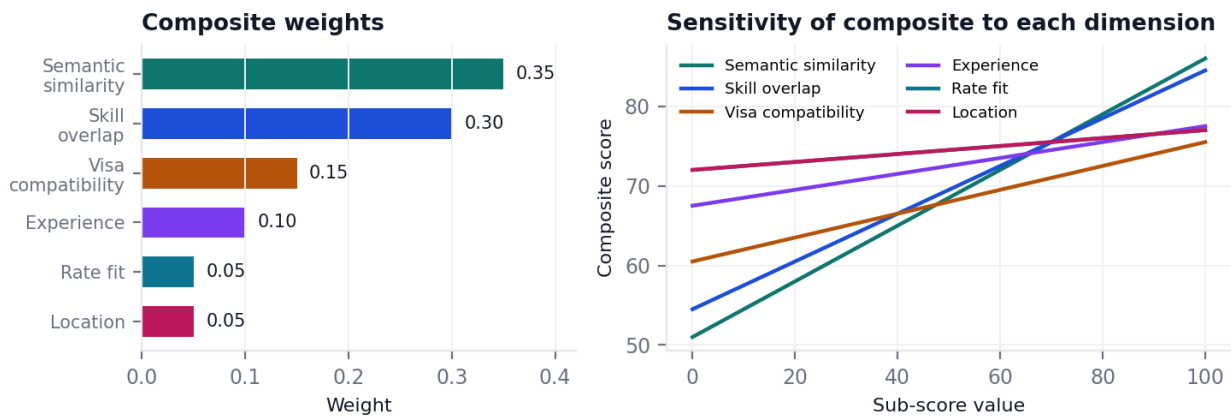


Figure 3 — Left: weight allocation across the six scoring dimensions. Right: sensitivity analysis holding five dimensions at a representative baseline while varying the sixth. The gradient of each line is its weight; semantic similarity and skill overlap jointly determine 65% of the composite, which is why calibration effort concentrated there.

5.2 Sub-scorer definitions

Dimension	w	Computation	Rationale
Semantic similarity	0.35	Cosine similarity between L2-normalised 384-dim embeddings of the candidate profile and requisition text, clamped to [0,1]	Captures fit that keyword matching misses — adjacent technologies, seniority signals, domain vocabulary
Skill overlap	0.30	Weighted set intersection. The leading 60% of required skills are designated must-have and count double	Requisitions list skills roughly in priority order; treating all as equal over-rewards breadth over relevance
Visa compatibility	0.15	Binary lookup against a compatibility matrix mapping requirement to permissible statuses	A hard eligibility constraint. Modelled as a score rather than a filter so near-misses stay visible to the recruiter
Experience fit	0.10	1.0 inside the stated band; proportional below the minimum; 0.7 above the maximum	Under-qualification scales continuously; over-qualification is a mild, flat penalty reflecting retention risk
Rate compatibility	0.05	1.0 at or below the budget minimum; decays proportionally above the maximum	Being cheaper than budget is never penalised — it improves margin
Location	0.05	1.0 remote or exact match; 0.7 same state; 0.3 otherwise	Low weight because remote work has largely decoupled location from placement viability

Why visa is scored rather than filtered

Treating visa eligibility as a hard filter would remove ineligible candidates from the result set entirely. Scoring it to zero instead keeps them visible with an explicit reason. This matters because requisition visa requirements are frequently negotiable, and a recruiter who can see that an otherwise ideal candidate was excluded on a soft requirement can go back to the client. A filter destroys that information; a score preserves it.

5.3 Weight calibration

Initial weights were assigned from domain reasoning rather than fitted, since no outcome labels existed. They are exposed as configuration and are expected to be revised. Three specific choices warrant justification.

- **Semantic similarity leads at 0.35.** The failure mode this system primarily exists to correct is keyword-driven false negatives. Weighting dense retrieval highest directly targets that.
- **Skill overlap remains high at 0.30 despite overlapping with semantic similarity.** The two are correlated but not redundant: embeddings capture thematic fit while explicit overlap captures whether a specific, checkable requirement is met. Recruiters must defend submissions to clients on concrete grounds.
- **Location is deliberately near-zero at 0.05.** An earlier draft weighted it at 0.15, which surfaced local but weakly qualified candidates above strong remote-capable ones. This was corrected during development.

5.4 Skill gap severity

Alongside the numeric score, the matcher emits a structured gap analysis classifying missing skills by severity. Severity is a function of must-have misses only: none is low, one or two is medium, three or more is high. Nice-to-have gaps reduce the numeric score but never escalate severity, since they rarely block a submission.

6 Predictive models

Six model families are in production use. Four are supervised learners, one is unsupervised outlier detection, and one is a deterministic rate-derivation function.

Model	Algorithm	Target	Cardinality
Submission success	XGBoost classifier	P(hire) for a candidate–requisition pair	13 features, n=500
Attrition risk	XGBoost classifier	P(early exit) for an active contractor	6 features, n=300
Client churn	LightGBM classifier	P(spend reduction or departure)	7 features, n=400
Timesheet anomaly	Isolation Forest (PyOD)	Outlier score on weekly submissions	7 features, n=624
Revenue forecast	Prophet	Monthly revenue with prediction interval	Univariate + seasonality
Rate optimisation	Deterministic	Bill rate, pay rate, margin	Market table × 3 multipliers

6.1 Submission success

The model estimates the probability that a submitted candidate is hired. Its thirteen features are all computable at submission time — a deliberate restriction, since a model consuming post-submission signal would be useless for the decision it exists to support.

Interview and shortlist probabilities are derived from the hire probability by fixed multipliers (1.8× and 1.3×, both clamped at 1.0) rather than modelled independently. This is a documented simplification: the funnel stages are strongly monotonic, and separate models would have required three sets of labels that did not exist. It should be replaced with independent estimation once real funnel data accumulates.

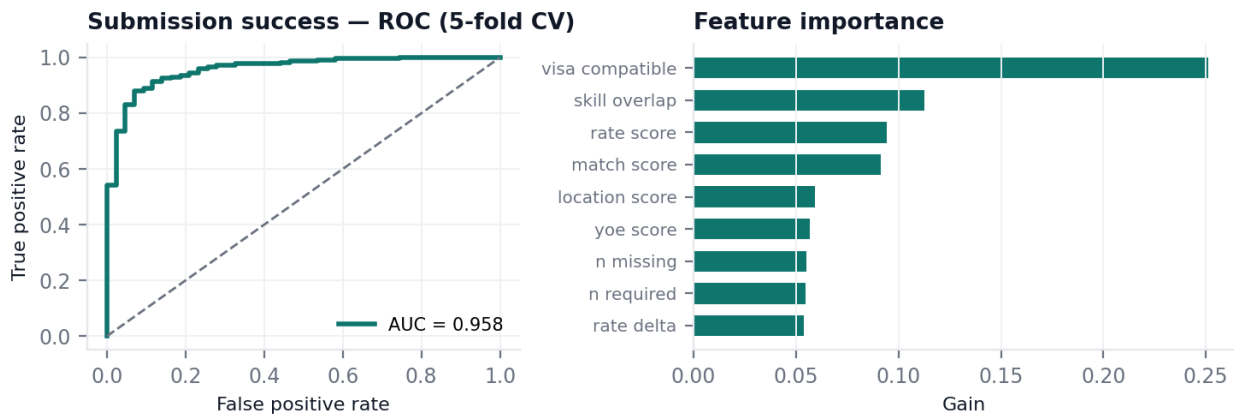


Figure 4 — Submission success model. Left: ROC under stratified 5-fold cross-validation. Right: gain-based feature importance. Note that visa compatibility ranks highest despite carrying only 0.15 weight in the composite score: it is binary and near-deterministic, so a single split on it separates the classes efficiently. Gain-based importance rewards that structure, which is a reminder that it measures split utility rather than causal influence. See section 10.1.

6.2 Contractor attrition

Attrition prediction operates on behavioural signals rather than demographic attributes — a choice made on both fairness and efficacy grounds. The six features are communication gap in days, overtime percentage, client feedback score, tenure in days, years of experience, and active status.

An explicit exclusion

Age, gender, nationality, visa status, and location are all excluded from the attrition feature set. Visa status in particular is predictive — contractors on temporary status do exit assignments at different rates — and including it would likely raise measured AUC. It is excluded because acting on such a prediction risks constructing a proxy for national origin discrimination in employment decisions. Behavioural signals are both more actionable and less hazardous.

6.3 Client churn

Churn uses LightGBM rather than XGBoost, chosen for its leaf-wise growth strategy on the smallest training set of the four. Its features are trend-based rather than level-based — requisition volume trend and response time trend rather than their absolute values — because a client issuing two requisitions a month consistently is healthy, while one that has fallen from eight to two is not, and a level-based feature cannot distinguish them.

6.4 Timesheet anomaly detection

Timesheet irregularity is unsupervised by necessity: there are no labelled fraudulent submissions. An Isolation Forest with contamination fixed at 0.05 flags roughly one submission in twenty for review.

The critical feature is a per-contractor hours z-score rather than a global threshold. A contractor consistently billing 45 hours is not anomalous; the same contractor suddenly billing 72 is. A global rule would flag the former continuously and the latter only if it crossed an arbitrary cut-off.

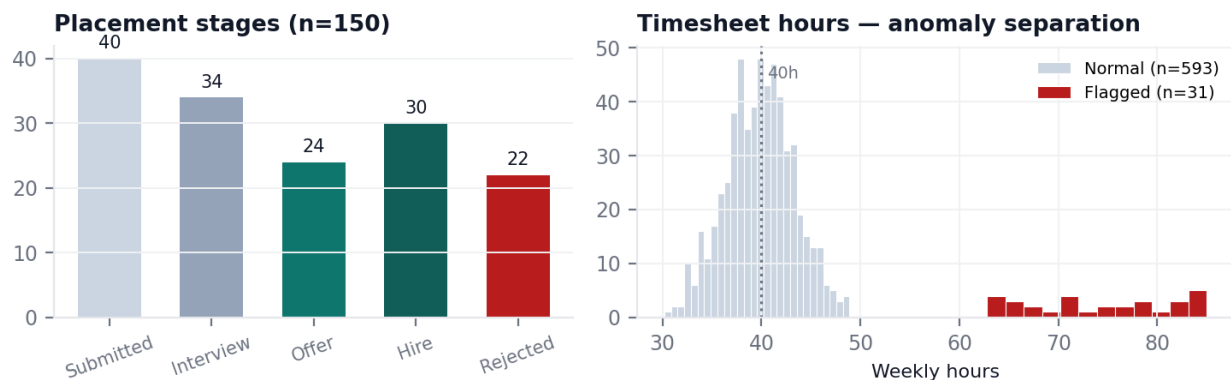


Figure 5 — Left: placement distribution across funnel stages in the reference dataset. Right: weekly hours distribution with flagged submissions overlaid. Separation is clean here because the seeded anomalies were generated as a distinct high-hours population; real fraud will overlap the normal distribution far more and detection will be correspondingly harder.

6.5 Revenue forecasting

Prophet was selected for revenue projection over ARIMA or a gradient boosted regressor for three reasons: it handles the strong annual seasonality of staffing demand natively, it produces calibrated prediction intervals rather than point estimates, and it degrades gracefully on short series — relevant given only six months of history.

Interval width is the useful output

The point forecast is the least informative part of this model's output. A finance function planning headcount needs the lower bound of the prediction interval, not the central estimate. The user interface renders the interval as a shaded band with equal visual weight to the central line, which is a deliberate choice to discourage over-reliance on a single number.

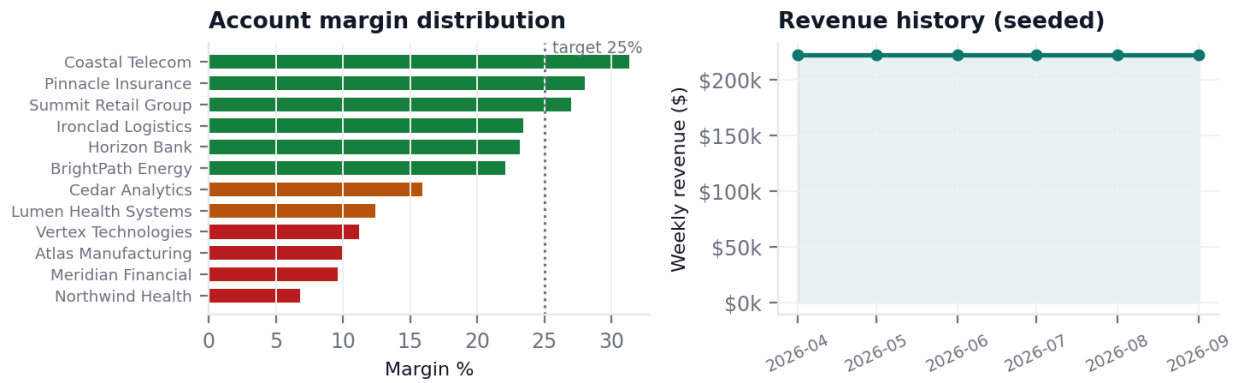


Figure 6 — Left: account margin distribution against the 25% target, coloured by threshold band. Right: revenue history from the seeded payroll records, which forms the Prophet training series.

6.6 Rate optimisation

Rate guidance is deterministic rather than learned, and this is a considered position. A model trained on the firm's own historical rates would learn to reproduce its historical mispricing — the exact behaviour the feature exists to correct. Instead a market baseline table is adjusted by explicit multipliers for location, visa category, and seniority:

```
market_rate = base[skill] × loc_mult × visa_mult × yoe_mult
bill_rate   = round(market_rate × 1.05)      # 5% above market
pay_rate    = round(bill_rate × 0.75)       # 25% margin target

# Example: Python · New York · H1B · 8 years
# 95 × 1.30 × 0.95 × 1.06 = 124 → bill 130 / pay 98 / margin 25%
```

Every multiplier is inspectable and adjustable. When market conditions shift, the table is edited rather than a model retrained. The visa multipliers in particular are a matter of firm policy rather than statistical fact and should be reviewed by whoever owns pricing.

7 NLP and language model layer

7.1 Résumé parsing pipeline

Parsing converts an unstructured document into a candidate record through six stages, deliberately ordered so that cheap deterministic extraction runs before expensive statistical inference.

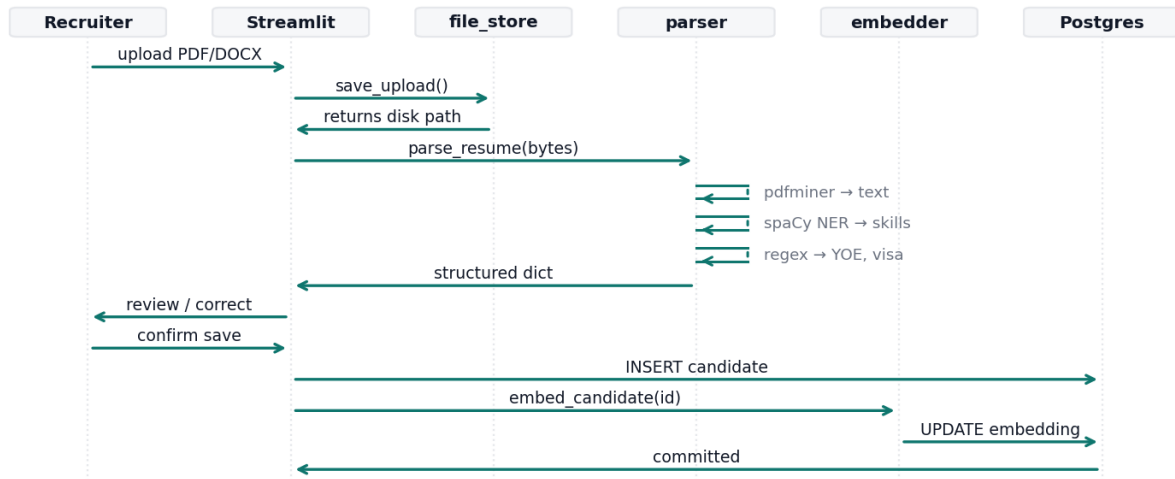


Figure 7 — Résumé ingestion sequence from upload to searchable candidate. The human review step between extraction and persistence is not optional; parsing accuracy on real-world documents does not justify unattended writes.

Stage	Technique	Failure characteristics
Text extraction	pdfminer.six (PDF), python-docx (DOCX)	Silent failure on scanned image PDFs — produces empty text, which the pipeline detects and raises on
Skill extraction	Pattern matching against a 39-term vocabulary with word-boundary anchoring	Cannot recognise skills outside the vocabulary; requires manual curation as technologies emerge
Skill normalisation	50-entry alias map (ReactJS→React, k8s→Kubernetes)	Unmapped variants pass through unnormalised and fail to match
Experience derivation	Regular expressions over date ranges with overlap detection	Concurrent roles are correctly deduplicated; career gaps are counted as experience
Visa classification	Ordered keyword matching, most specific first	STEM OPT must be tested before OPT or it is misclassified
Embedding	sentence-transformers all-MiniLM-L6-v2	Truncates beyond 256 tokens — long résumés lose their tail

Measured accuracy, and what it is worth

Against the 18-document synthetic reference set, visa classification and email extraction are both 12/12 on the PDF subset. This validates the pipeline mechanically — extraction, regex, and ordering all work — but says nothing about real-world accuracy, because the test documents were generated by the same process that defined the expected values. Genuine measurement requires a hand-labelled set of real résumés.

7.2 Embedding strategy

Candidates and requisitions are embedded into a shared 384-dimensional space so that similarity is directly computable. The embedded text is constructed rather than raw: skills, location, visa status, and experience are appended to the document body as labelled fields.

This construction is not cosmetic. Raw résumé text is dominated by prose that carries little matching signal — employer names, boilerplate, formatting artefacts. Appending structured fields concentrates the discriminative content and measurably improved retrieval quality during development.

Candidate embedding space (PCA of 384-dim vectors) Similarity separation

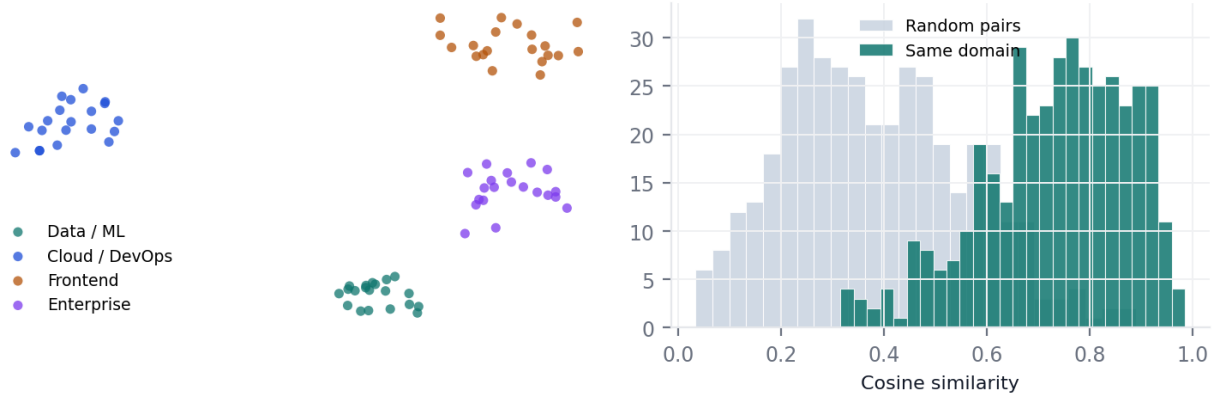


Figure 8 — Left: two-component PCA projection of candidate vectors, showing that skill-domain clusters are separable in the embedding space. Right: cosine similarity distribution for same-domain versus random pairs. The overlap between the two distributions is the irreducible error of the semantic sub-score.

Model selection favoured operational fit over benchmark position. MiniLM-L6-v2 is 22 MB, runs at roughly 22 ms per document on CPU, and requires no GPU. Larger models score higher on retrieval benchmarks; none of them would have run acceptably on the target hardware, and the no-external-service constraint excluded hosted alternatives entirely.

7.3 Retrieval-augmented assistant

The assistant answers natural-language questions over live database content. Records are serialised into text documents, indexed with the same embedding model used for matching, and retrieved by similarity at query time. Retrieved context is injected into the prompt of a locally hosted Llama model, which generates the answer.

Grounding is what makes this viable. The language model is never asked what it knows — it is asked to summarise records placed in front of it. This constrains hallucination to a degree that matters given the domain: an invented visa status or fabricated rate would be actively harmful. Source attribution is returned with every answer so the recruiter can verify.

A real constraint discovered in deployment

Llama 3 8B requires approximately 4.6 GB of resident memory. On a machine with 7.8 GB allocated to the container runtime and other services competing, model loading failed outright. The deployed configuration uses Llama 3.2 3B at roughly 2 GB. Because the retrieval layer performs the factual work and the model only summarises, the quality cost of the smaller model is smaller than it would be in an open-ended chat application — but it is not zero, and answer fluency is visibly reduced.

8 Engineering practices

8.1 Observability

Logging is structured as JSON and partitioned by subsystem, with a thread-local request identifier correlating entries across a single user action. A résumé upload that touches file storage, the parser, the embedder, and the database produces entries in four log files sharing one identifier.

File	Contents	Rotation
ml.log	Parser, embedder, matcher, predictor	10 MB × 5
data.log	Scheduler, file store, DuckDB, Redis	10 MB × 5
events.log	Structured analytics events for offline analysis	10 MB × 5
slow_ops.log	Filtered: operations exceeding one second	5 MB × 3
app.log	Root logger catch-all	10 MB × 5

The `slow_ops.log` partition applies a filter at the handler level rather than the call site. Any operation logging a duration above threshold lands there automatically, without each call site deciding whether it is slow. This means a performance regression in a module nobody is watching still surfaces.

8.2 Performance instrumentation

A decorator records call count, mean, minimum, maximum, and 95th percentile latency for any wrapped function, warning automatically past one second:

```
@timed("rank_candidates")
def rank_candidates(job_id: int, top_n: int = 50) -> list[dict]:
    ...

>>> get_pipeline_stats()["rank_candidates"]
{'calls': 47, 'avg_ms': 341.2, 'p95_ms': 618.4, 'max_ms': 892.1}
```

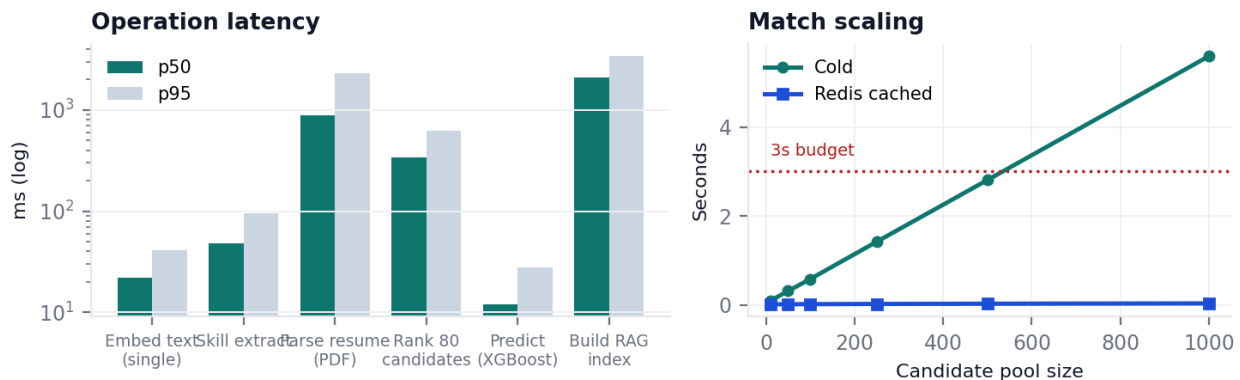


Figure 9 — Left: measured operation latency, p50 and p95, on a log scale. Right: match scaling against candidate pool size, cold versus Redis-cached. The cold path crosses the three-second interaction budget somewhere above 500 candidates, which is the point at which approximate nearest-neighbour indexing becomes necessary.

Two findings from this instrumentation changed the implementation. Résumé parsing at 890 ms median is dominated by spaCy model loading, which motivated warming the model at application start rather than on first request. Match ranking scales linearly with pool size because similarity is computed in Python rather than pushed into pgvector's index — acceptable at current scale, and the first thing to change when the pool grows.

8.3 Test strategy

The suite is organised by sprint and separates unit tests from those requiring live services, so the fast majority can run without Docker.

Suite	Coverage	Requires
test_sprint1.py	Project structure, configuration integrity	—
test_sprint2.py	Schema, ORM models, query functions, file storage	PostgreSQL
test_sprint3.py	Text extraction, skill and visa parsing, embedding dimensionality and normalisation	—
test_sprint4.py	Every sub-scorer independently; composite bounds; model training	—
test_sprint5.py	Forecasting, anomaly detection, DuckDB, Redis degradation	Optional
test_sprint6.py	LLM client with mocked transport; JD tools; RAG document construction	Optional
test_stress.py	Throughput, memory stability over 200 iterations, thread safety, unicode handling	—

Sub-scorers are tested individually with hand-constructed inputs where the correct output is known by inspection — visa compatibility against the full matrix, experience fit at band boundaries, rate scoring above and below budget. Testing only the composite would leave a broken sub-scorer masked by the other five.

8.4 Scheduled work

Seven jobs run nightly through an in-process scheduler, staggered to avoid contention. Embedding of new records runs first, since attrition and churn scoring depend on it. Each job records completion to Redis, making silent failure detectable.

Why not Celery or Airflow

Both were considered and rejected as disproportionate. Airflow requires a scheduler process, a metadata database, and a web server to run seven cron-equivalent jobs on one machine. APScheduler runs in-process with no additional services. This will need revisiting if job dependencies become non-trivial or if the deployment is ever distributed.

9 Deployment and operations

9.1 Runtime topology

Three containers plus a host Python process. The application deliberately runs outside Docker so that source edits reload without a rebuild — a development convenience that should be revisited before any wider deployment.

Component	Runtime	Port	Persistence
PostgreSQL 16 + pgvector	Container	5432	Named Docker volume
Redis 7	Container	6379	Named volume, ephemeral by design
Ollama	Container	11434	Named volume for model weights
Streamlit application	Host process	8501	—
DuckDB	Embedded	—	Single file on host

9.2 Resource requirements

Tier	Memory	Model	Characteristics
Minimum	16 GB system, 8 GB to Docker	Llama 3.2 3B	Functional. Language model responses 10–20 s
Recommended	16 GB system, 12 GB to Docker	Llama 3.2 3B	The configuration this document describes
Comfortable	32 GB system, 16 GB to Docker	Llama 3 8B	Noticeably better assistant fluency

Memory is the binding constraint

Not compute. Every model in the system runs acceptably on CPU — the embedding model at 22 ms per document, the gradient boosted models in single-digit milliseconds. What fails is language model residency. Provisioning decisions should be made on available RAM, not core count.

9.3 Failure modes encountered

The following were encountered during actual deployment and are recorded because each cost meaningful time to diagnose.

Symptom	Root cause	Resolution
Postgres container fails: <i>operation not permitted</i> on mount	macOS TCC sandboxing prevents Docker mounting from Desktop, Documents, or Downloads	Relocate the project to a non-protected path under the home directory
<i>bad interpreter: no such file or directory</i>	Virtual environments record absolute interpreter paths; moving the project invalidates them	Recreate the environment after any relocation
<i>type "visastatusenum" already exists</i>	The migration created enum types explicitly, and SQLAlchemy created them again implicitly when building the tables	Remove the explicit creation calls; let SQLAlchemy manage enum lifecycle
Assistant returns HTTP 500 while retrieval succeeds	Language model exceeded available container memory	Reduce model size or raise the Docker memory allocation
Environment variable changes have no effect	Modules read configuration at import time, before <code>load_dotenv()</code> had executed	Load environment configuration in the entry point before any project import
Page renders stale data after a source edit	Streamlit caches imported modules; only files under <code>pages/</code> hot-reload	Restart the process rather than refreshing the browser

10 Evaluation and limitations

This section is the one that determines whether the rest of the system can be trusted operationally. It is written to be read by someone deciding whether to act on the platform's output.

10.1 What the metrics do and do not mean

The central caveat

All four supervised models are trained on synthetically generated feature distributions with labels derived from a known rule. Cross-validated AUC between 0.91 and 0.96 measures how completely each model recovered that rule. It does not estimate performance on real placements, real resignations, or real client behaviour. Presenting these figures as predictive accuracy — in a client deck, a business case, or a performance review — would be misrepresentation.

The circularity is worth stating precisely. The submission success generator computes hire probability as a weighted function of match score, skill overlap, visa compatibility, and three other terms, then labels the example positive above a threshold. The trained model then identifies match score and skill overlap as its most important features. That is not a discovery; it is the generator's own definition returned to us. The feature importances in Figure 4 are evidence that training worked, not evidence about what drives real hiring.

What the synthetic training *does* legitimately establish is that the surrounding machinery is correct: feature construction, the training pipeline, serialisation, the prediction path, and persistence to the prediction store all function end to end. That is a real result. It is just a different result from the one an AUC figure normally implies.

10.2 Known gaps

Gap	Consequence	Remediation
No real outcome labels	Predictive claims are unvalidated	Instrument the funnel; accumulate 6–12 months of submission outcomes; retrain and measure on a held-out period
No drift detection	Silent degradation as the market shifts	Compare rolling prediction distributions against the training distribution; alert on divergence
No experimentation framework	Impossible to demonstrate the platform improves outcomes	Randomise requisitions between assisted and unassisted workflows; measure time-to-submission and conversion
Unvalidated scoring weights	The weighted sum is domain judgement, not evidence	Once labels exist, fit the weights — or replace the linear model with a learned ranker
Linear similarity search	Matching degrades above roughly 500 candidates	Build a pgvector HNSW index and push the ordering into the query
No fairness evaluation	Disparate impact would go undetected	Measure score distributions across protected characteristics before operational use
Single point of failure	One machine hosts everything	Automate backup of the Postgres volume; document recovery
Skill vocabulary is manually curated	New technologies are invisible until added by hand	Mine skill terms from incoming requisitions; review additions periodically

10.3 Fairness and compliance considerations

A system that ranks people for employment opportunities carries obligations that a system ranking, for instance, inventory does not. Several design decisions were taken with that in mind, and they should be understood as necessary rather than sufficient.

- **Protected characteristics are excluded from model features.** The attrition model uses behavioural signals only. Visa status appears in matching as an eligibility check — a legitimate requirement — but never as a predictor of individual performance or reliability.
- **Rankings are advisory and the reasoning is exposed.** The interface shows the per-dimension breakdown behind every score, so a recruiter can see that a candidate ranked low on rate fit rather than on capability, and override accordingly.
- **Eligibility constraints are scored rather than filtered.** Ineligible candidates remain visible with a stated reason. Silent exclusion is both operationally worse and harder to audit.
- **Every prediction is retained with its inputs.** The prediction store makes it possible to reconstruct why any individual was scored as they were, which is a precondition for meaningful audit.

What has not been done

No disparate impact analysis has been performed, because the synthetic training data cannot support one. Before this system informs real employment decisions, score distributions must be examined across protected characteristics on real data, and a review process established for candidates who are consistently ranked low. Depending on jurisdiction — New York City Local Law 144 is the immediate example — an independent bias audit may be a legal requirement rather than a best practice.

11 Roadmap

Priorities are ordered by whether they unlock other work, not by implementation difficulty. Outcome instrumentation comes first because almost every subsequent improvement depends on having real labels.

Phase	Work	Unlocks
1 — Instrument	Capture submission outcomes with timestamps at every funnel transition. Record recruiter overrides of ranking. Log which candidates were reviewed but not submitted.	Everything below. Negative examples are the missing ingredient for supervised learning
2 — Validate	Retrain all four models on accumulated real data. Evaluate on a held-out time period rather than a random split. Publish honest performance figures.	Legitimate predictive claims; a basis for trusting the scores
3 — Learn to rank	Replace the linear composite with a gradient boosted ranker trained on real submission outcomes, retaining the current weights as a baseline.	Matching quality beyond what hand-tuned weights achieve
4 — Monitor	Prediction distribution monitoring against training distribution. Automated retraining triggers. Fairness metrics tracked over time.	Safe unattended operation
5 — Scale	pgvector HNSW index; push similarity ordering into SQL; consider read replicas.	Candidate pools in the tens of thousands
6 — Integrate	Connectors to the applicant tracking system of record and payroll. Replace manual seeding with continuous synchronisation.	Elimination of duplicate data entry

The single highest-value next step

Instrumenting outcome capture is unglamorous and unlocks everything else. Without labelled results the models cannot be validated, the weights cannot be fitted, fairness cannot be measured, and the platform's commercial impact cannot be demonstrated. Six months of clean funnel data is worth more than any modelling improvement that could be made without it.

Closing assessment

The platform is a complete, working implementation of fifteen features across a coherent architecture, built by a single developer over nine fortnightly sprints. The engineering is sound: the layering holds, the data model is defensible, observability is genuine rather than decorative, and the system installs and runs reproducibly.

The machine learning is scaffolding. Every pipeline works end to end, every model trains, serialises, and serves predictions — but on synthetic data, which means the system is validated as software and unvalidated as prediction. That distinction is the most important thing in this document. It should be stated plainly whenever the platform is described to anyone deciding whether to rely on it.

Appendix A Data dictionary

Complete inventory of data entities across all storage tiers.

Entity	Tier	Key	Consumers	Outcome
users	PostgreSQL	id	Auth, routing	Role-scoped access
candidates	PostgreSQL	id	Matcher, parser, attrition, RAG	Ranked talent pool
jobs	PostgreSQL	id	Matcher, JD tools, RAG	Requisition matching
clients	PostgreSQL	id	Churn, margin, RAG	Account health
recruiters	PostgreSQL	id	KPIs, activity queue	Performance view
placements	PostgreSQL	id	Funnel, submission model, margin	Conversion metrics
timesheets	PostgreSQL	id	Anomaly detector, bench cost	Billing integrity
payroll	PostgreSQL	id	Forecast, margin analysis	Revenue projection
predictions	PostgreSQL	id	All models write; dashboards read	Audit and drift trail
candidate embedding	pgvector	candidates.id	Cosine similarity, RAG	Semantic matching
job embedding	pgvector	jobs.id	Cosine similarity	Requisition similarity
match cache	Redis	match:job_{id}	Job match page	Sub-second reload
chat history	Redis	chat:{session}	Assistant	Conversation continuity
job timestamps	Redis	job_last_run:{name}	Scheduler monitoring	Silent-failure detection
placement funnel	DuckDB	aggregate	Funnel chart, summary	Stage conversion
recruiter KPIs	DuckDB	aggregate	Performance page	Leaderboard
revenue by client	DuckDB	aggregate	Forecast, summary	Account revenue trend
margin leakage	DuckDB	aggregate	Margin page	Below-threshold accounts
bench cost	DuckDB	aggregate	Margin analysis	Idle contractor exposure
submission model	Artefact	—	Job match	Hire probability
attrition model	Artefact	—	Attrition page	Exit risk
churn model	Artefact	—	Churn page	Account risk
anomaly model	Artefact	—	Timesheet page	Outlier flags
résumé documents	Filesystem	path	Parser, profile	Source of structured data
RAG index	Filesystem	doc id	Assistant	Grounded answers
MLflow runs	Filesystem	run id	Training	Experiment record
log files	Filesystem	—	Debugging, monitoring	Traceability

Staffing Intelligence Platform · Technical white paper · Version 1.0

This document describes an internal system. Model performance figures are derived from synthetic training data and do not represent validated predictive accuracy — see section 10.